Project Design Phase-II
Solution Requirements (Functional & Non-functional)

Date	4 March 2026
Team ID	
Project Name	PlantCare AI: Advanced Plant Disease Detection Using Transfer Learning
Maximum Marks	4 Marks

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User Registration	Registration through Form Registration through Gmail Registration through LinkedIN
FR-2	User Confirmation	Confirmation via Email Confirmation via OTP
FR-3	Image Upload & Processing	Upload leaf image via web/mobile interface Validate file type (allow only JPEG/PNG) Resize and normalize image to 224x224 pixels
FR-4	Disease Prediction (ML Engine)	Pass preprocessed image to MobileNetV2 model Classify image into one of 38 disease classes Calculate and return prediction confidence score
FR-5	Dashboard & History	Display history of past user uploads and diagnoses Export diagnostic history as a CSV/PDF report
FR-6	Notification System	Display real-time diagnostic results on the UI Send automated email summary of the diagnostic report

Non-functional Requirements:

Following are the non-functional requirements of the proposed solution.

FR No.	Non-Functional Requirement	Description
NFR-1	**Usability**	The web interface must be intuitive, mobile-responsive, and simple enough for non-technical users (like home gardeners) to navigate easily.
NFR-2	**Security**	User passwords must be hashed. Uploaded images stored in S3 and database logs must be encrypted at rest (e.g., using AWS KMS).

NFR-3	**Reliability**	The system must handle invalid inputs gracefully (e.g., rejecting .txt or .pdf files without crashing the server).
NFR-4	**Performance**	The ML model inference and server response delivery should be completed in under 2 seconds per image upload to ensure a fast user experience.
NFR-5	**Availability**	The web application and API should be hosted on a cloud platform ensuring 99.9% uptime, especially critical during peak agricultural seasons.
NFR-6	**Scalability**	The backend architecture must be decoupled so the Flask API can scale horizontally to handle hundreds of concurrent user image uploads if needed.